

Hướng dẫn thực hiện bài tập lớn

Học phần Dịch vụ kết nối và Công nghệ nền tảng

ThS. Lê Thái Bảo, ThS. Lê Thị Thùy Trang

2026-05-02

1 Giới thiệu bài tập lớn

1.1 Bối cảnh

Bài tập lớn học phần **Dịch vụ kết nối và Công nghệ nền tảng** được thiết kế theo định hướng **project-based learning** và **OBE**. Sinh viên không chỉ học từng công cụ riêng lẻ, mà sẽ cùng xây dựng một hệ thống gồm nhiều service có khả năng kết nối với nhau.

Trong học phần này, mỗi nhóm phụ trách một service độc lập. Tuy nhiên, service đó không được tồn tại một mình. Service phải có đầu vào, đầu ra, hợp đồng API, kiểm thử, đóng gói Docker và khả năng kết nối với service khác.

Nói ngắn gọn:

Không chỉ làm cho “chạy được trên máy mình”, mà phải làm sao để service của nhóm mình **kết nối được với service của nhóm khác**.

Case study xuyên suốt của học phần là **Smart Campus Operations Platform** – một nền tảng mô phỏng hệ thống vận hành thông minh trong khuôn viên trường đại học. Hệ thống có thể nhận dữ liệu từ thiết bị IoT, camera, cổng ra vào, xử lý bằng AI, sinh cảnh báo và gửi thông báo đến người phụ trách.

1.2 Tinh thần thực hiện

Bài tập lớn được thiết kế theo tư duy tích hợp hệ thống. Sinh viên cần chứng minh được rằng mình hiểu cách một service hiện đại được thiết kế, kiểm thử, đóng gói và đưa vào một hệ thống lớn hơn.

Một service tốt trong học phần này cần thỏa mãn các yêu cầu sau:

- Có **service boundary** rõ ràng: biết mình nhận gì, trả gì, gọi ai và ai gọi mình.
- Có **hợp đồng API** rõ ràng bằng `openapi.yaml`.
- Có **kiểm thử tích hợp** bằng Postman hoặc Newman.
- Có **Dockerfile** để đóng gói môi trường chạy.
- Có **docker-compose.yml** nếu service cần database, worker hoặc thành phần phụ trợ.
- Có khả năng **kết nối với ít nhất một service khác**.
- Có **minh chứng vận hành**: log, screenshot, video demo hoặc test report.

Nhóm không được chỉ nộp source code. Bài tập lớn chỉ được nghiệm thu khi có đủ minh chứng thiết kế, kiểm thử, đóng gói và kết nối.

Học phần này không chấm nặng giao diện. Front-end là cần có, nhưng không được thay thế cho API, OpenAPI, Postman test, Docker và minh chứng tích hợp.

2 Mục tiêu học tập

Sau khi hoàn thành bài tập lớn, sinh viên có thể:

1. Phân tích được một bài toán tích hợp hệ thống và xác định được ranh giới dịch vụ.
2. Phân biệt được actor, consumer, provider, service boundary và platform component.
3. Thiết kế được hợp đồng API bằng OpenAPI/Swagger.
4. Xây dựng được service có endpoint, request, response, status code và error model rõ ràng.
5. Xây dựng được bộ kiểm thử tích hợp bằng Postman/Newman.
6. Đóng gói được service bằng Docker.
7. Cấu hình được service chạy cùng database hoặc thành phần phụ trợ bằng Docker Compose.
8. Kết nối được service của nhóm mình với service của nhóm khác.
9. Giải thích được quyết định kiến trúc, công cụ và phương pháp triển khai của nhóm.
10. Trình bày được Demo Pack cuối kỳ dựa trên minh chứng vận hành thực tế.

3 Quy định chung

3.1 Tổ chức nhóm

Lớp được chia thành **14 nhóm**. Trong đó:

- Có **7 loại service**.
- Mỗi loại service có **2 nhóm cùng làm**.
- 7 service kết nối với nhau tạo thành **1 sản phẩm dịch vụ hoàn chỉnh**.
- Vì có 14 nhóm nên cả lớp sẽ tạo ra **2 sản phẩm song song**: Product A và Product B.

Mỗi nhóm nên có 3–4 sinh viên. Các service nhiều phụ thuộc hơn như **Core Business**, **AI Vision** và **Analytics** có thể ưu tiên nhóm 4 sinh viên nếu cần.

3.2 Nguyên tắc hai nhóm làm cùng một service

Hai nhóm có thể cùng làm một loại service, nhưng thuộc hai sản phẩm khác nhau. Ví dụ:

- Nhóm A1 làm **IoT Ingestion** cho Product A.
- Nhóm B1 làm **IoT Ingestion** cho Product B.

Hai nhóm cùng tên service **không bắt buộc dùng cùng công cụ, cùng ngôn ngữ lập trình hoặc cùng phương pháp triển khai**.

Ví dụ:

- Nhóm A1 làm IoT Ingestion bằng Python và FastAPI.
- Nhóm B1 làm IoT Ingestion bằng Node.js và Express.

Miền là service đáp ứng đúng hợp đồng API, chạy được, test được và kết nối được với hệ thống.

3.3 Công cụ tự chọn, hợp đồng API bắt buộc thống nhất

Giảng viên không bắt buộc một stack duy nhất cho toàn lớp. Sinh viên có thể chọn:

- Python hoặc Node.js.
- FastAPI, Flask, Express hoặc Fastify.
- SQLite, PostgreSQL, MongoDB hoặc file-based storage ở mức mô phỏng.
- REST API hoặc kết hợp message queue nếu nhóm làm được.

Tuy nhiên, dù chọn công cụ nào, nhóm vẫn phải đảm bảo:

1. Có API rõ ràng.
2. Có `openapi.yaml`.
3. Có test bằng Postman/Newman.
4. Có Dockerfile.
5. Có `docker-compose.yml` nếu cần.
6. Có thể kết nối với service khác.
7. Có minh chứng chạy được.

Công cụ có thể khác nhau, nhưng hợp đồng kết nối phải thống nhất.

4 Bối cảnh sản phẩm

4.1 Smart Campus Operations Platform

Smart Campus Operations Platform là nền tảng mô phỏng hệ thống vận hành thông minh trong khuôn viên trường học. Hệ thống gồm 7 service chính:

1. IoT Ingestion Service.
2. Camera Stream Service.
3. Access Gate Service.
4. AI Vision Service.
5. Analytics Service.
6. Core Business Service.
7. Notification Service.

Luồng dữ liệu tổng quát:

```

ESP32 / Sensor
↓
IoT Ingestion Service
↓
Core Business Service
↓
Notification Service

```

```

Camera IP
↓
Camera Stream Service
↓
AI Vision Service
↓
Core Business Service
↓
Notification Service

```

```

RFID Reader
↓
Access Gate Service
↓
Core Business Service

```

```

IoT / Camera / Access / Core
↓
Analytics Service
↓
Báo cáo, thống kê, metric

```

Một sản phẩm được xem là hoàn chỉnh khi 7 service có thể phối hợp để trả lời các câu hỏi vận hành như:

- Có người lạ xuất hiện trong khu vực không?
- Có lượt ra/vào bất thường không?
- Nhiệt độ, độ ẩm, chuyển động có vượt ngưỡng không?
- Khi có sự kiện bất thường, hệ thống có gửi thông báo không?
- Có thống kê được số lượt truy cập, số cảnh báo, trạng thái cảm biến không?

5 Phân chia đề tài bài tập lớn

5.1 Bảng phân chia chính thức

Mã nhóm	Sản phẩm	Service phụ trách	Tên đề tài
A1	Product A	IoT Ingestion	Xây dựng dịch vụ tiếp nhận dữ liệu IoT.
A2	Product A	Camera Stream	Xây dựng dịch vụ tiếp nhận và xử lý luồng camera.
A3	Product A	Access Gate	Xây dựng dịch vụ kiểm soát ra/vào.
A4	Product A	AI Vision	Xây dựng dịch vụ AI phân tích hình ảnh.
A5	Product A	Analytics	Xây dựng dịch vụ tổng hợp và phân tích dữ liệu.
A6	Product A	Core Business	Xây dựng dịch vụ xử lý nghiệp vụ trung tâm.
A7	Product A	Notification	Xây dựng dịch vụ gửi cảnh báo đa kênh.
B1	Product B	IoT Ingestion	Xây dựng dịch vụ tiếp nhận dữ liệu IoT.
B2	Product B	Camera Stream	Xây dựng dịch vụ tiếp nhận và xử lý luồng camera.
B3	Product B	Access Gate	Xây dựng dịch vụ kiểm soát ra/vào.
B4	Product B	AI Vision	Xây dựng dịch vụ AI phân tích hình ảnh.
B5	Product B	Analytics	Xây dựng dịch vụ tổng hợp và phân tích dữ liệu.
B6	Product B	Core Business	Xây dựng dịch vụ xử lý nghiệp vụ trung tâm.
B7	Product B	Notification	Xây dựng dịch vụ gửi cảnh báo đa kênh.

5.2 Nguyên tắc chọn đề tài ở Buổi 1

Mỗi nhóm đăng ký đúng một service. Mỗi service có đúng hai nhóm làm, tương ứng với Product A và Product B.

Các service có mức độ phụ thuộc khác nhau:

Service	Độ phụ thuộc	Ghi chú
IoT Ingestion	Trung bình	Phù hợp nhóm muốn làm API, sensor data, event.
Camera Stream	Trung bình–khó	Phù hợp nhóm thích xử lý ảnh, camera, OpenCV.
Access Gate	Trung bình	Phù hợp nhóm muốn làm CRUD, rule đơn giản, log ra/vào.
AI Vision	Khó	Phù hợp nhóm muốn làm AI service, YOLO, image inference hoặc AI mock.
Analytics	Khó	Phù hợp nhóm thích dữ liệu, tổng hợp metric, báo cáo JSON.
Core Business	Khó nhất	Service trung tâm, cần làm việc với nhiều nhóm khác.
Notification	Trung bình	Phù hợp nhóm muốn làm Telegram, email, webhook, retry.

6 Mô tả chi tiết các đề tài

6.1 Đề tài 1. IoT Ingestion Service

6.1.1 Vai trò

Service này chịu trách nhiệm nhận dữ liệu từ thiết bị IoT như ESP32, cảm biến nhiệt độ, độ ẩm và cảm biến chuyển động.

6.1.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận dữ liệu cảm biến.
- Kiểm tra dữ liệu hợp lệ hay không.
- Lưu hoặc chuyển tiếp dữ liệu.
- Gửi event cho Core Business hoặc Analytics.

6.1.3 Dữ liệu đầu vào gợi ý

```
{
  "device_id": "esp32-a101",
  "temperature": 31.5,
  "humidity": 72,
  "motion": true,
  "timestamp": "2026-05-02T09:00:00"
}
```

6.1.4 Đầu ra mong đợi

- API nhận dữ liệu cảm biến.
- Event hoặc response cho biết dữ liệu đã được ghi nhận.
- Log hoặc dữ liệu để Analytics có thể tổng hợp.

6.1.5 Service cần kết nối

Kết nối với	Mục đích
Core Business	Gửi dữ liệu để kiểm tra ngưỡng bất thường.
Analytics	Cung cấp dữ liệu thống kê.
Notification	Gián tiếp thông qua Core khi có cảnh báo.

6.1.6 Công cụ gợi ý

- Python + FastAPI.
- Node.js + Express/Fastify.
- MQTT + HTTP API.
- SQLite, PostgreSQL hoặc MongoDB.
- Docker.

6.1.7 Artefact bắt buộc

```
- openapi.yaml
- service_boundary.md
- postman_collection.json
- Dockerfile
- .env.example
- RUN_LOCAL.md
- evidence/iot_ingestion_test_report.md
```

6.2 Đề tài 2. Camera Stream Service

6.2.1 Vai trò

Service này mô phỏng việc nhận dữ liệu từ camera, quản lý frame ảnh hoặc video stream, và gọi AI Vision khi phát hiện điều kiện cần phân tích.

6.2.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận ảnh hoặc video frame.
- Mô phỏng camera stream nếu không có camera thật.
- Xác định khi nào cần gửi ảnh sang AI Vision.

- Gửi request đến AI Vision Service.
- Nhận kết quả phát hiện từ AI Vision.

6.2.3 Dữ liệu đầu vào gợi ý

```
{
  "camera_id": "cam-gate-01",
  "frame_url": "http://example.com/frame.jpg",
  "motion_detected": true,
  "timestamp": "2026-05-02T09:05:00"
}
```

6.2.4 Đầu ra mong đợi

- API upload frame hoặc nhận frame URL.
- Trigger gọi AI Vision.
- Log kết quả phân tích camera.

6.2.5 Service cần kết nối

Kết nối với	Mục đích
AI Vision	Gửi ảnh/frame để nhận diện.
Core Business	Chuyển kết quả bất thường.
Analytics	Cung cấp dữ liệu camera event.

6.2.6 Công cụ gợi ý

- Python + FastAPI.
- OpenCV.
- Flask/FastAPI.
- Node.js nếu chỉ mô phỏng bằng ảnh tĩnh.
- Docker.

6.3 Đề tài 3. Access Gate Service

6.3.1 Vai trò

Service này mô phỏng hệ thống kiểm soát ra/vào bằng thẻ RFID, mã sinh viên, mã nhân viên hoặc QR code.

6.3.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận sự kiện quét thẻ hoặc nhập mã.

- Kiểm tra thông tin người dùng.
- Ghi log ra/vào.
- Gửi sự kiện sang Core Business để kiểm tra chính sách.

6.3.3 Dữ liệu đầu vào gợi ý

```
{
  "card_id": "RFID-2026-001",
  "gate_id": "gate-main",
  "direction": "IN",
  "timestamp": "2026-05-02T07:30:00"
}
```

6.3.4 Đầu ra mong đợi

```
{
  "access_granted": true,
  "reason": "Valid student card",
  "person_id": "SV001"
}
```

6.3.5 Service cần kết nối

Kết nối với	Mục đích
Core Business	Kiểm tra quyền truy cập.
Analytics	Cung cấp dữ liệu lượt vào/ra.
Notification	Gián tiếp thông qua Core nếu có truy cập bất thường.

6.3.6 Công cụ gợi ý

- Node.js + Fastify/Express.
- Python + FastAPI.
- PostgreSQL hoặc SQLite.
- Docker.

6.4 Đề tài 4. AI Vision Service

6.4.1 Vai trò

Service này chịu trách nhiệm xử lý ảnh bằng AI, ví dụ phát hiện người, phát hiện người lạ, nhận diện khuôn mặt ở mức mô phỏng hoặc phát hiện đối tượng.

6.4.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận ảnh từ Camera Stream.
- Chạy mô hình AI hoặc mô phỏng kết quả AI.
- Trả về kết quả phát hiện.
- Cho biết mức độ tin cậy của kết quả.

6.4.3 Dữ liệu đầu vào gợi ý

```
{
  "camera_id": "cam-gate-01",
  "image_url": "http://example.com/frame.jpg",
  "timestamp": "2026-05-02T09:10:00"
}
```

6.4.4 Đầu ra mong đợi

```
{
  "detected": true,
  "object": "person",
  "confidence": 0.91,
  "risk_level": "medium"
}
```

6.4.5 Service cần kết nối

Kết nối với	Mục đích
Camera Stream	Nhận ảnh/frame cần phân tích.
Core Business	Gửi kết quả để ra quyết định.
Analytics	Cung cấp dữ liệu thống kê phát hiện.

6.4.6 Công cụ gợi ý

- Python + FastAPI.
- YOLOv8, YOLOv11 hoặc OpenCV.
- Ultralytics.
- Mock AI nếu máy yếu, nhưng phải giải thích rõ.
- Docker.

6.5 Đề tài 5. Analytics Service

6.5.1 Vai trò

Service này chịu trách nhiệm tổng hợp dữ liệu từ nhiều service khác để tạo metric, thống kê và báo cáo.

6.5.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận hoặc lấy dữ liệu từ IoT, Camera, Access Gate và Core Business.
- Tổng hợp metric.
- Trả về báo cáo dạng JSON.
- Có thể cung cấp endpoint phục vụ dashboard.

6.5.3 Metric gợi ý

- Số lượt ra/vào theo giờ.
- Nhiệt độ trung bình theo phòng/khu vực.
- Số cảnh báo trong ngày.
- Số lần phát hiện chuyển động.
- Số event bất thường.

6.5.4 Đầu ra gợi ý

```
{
  "date": "2026-05-02",
  "total_access": 120,
  "total_alerts": 5,
  "avg_temperature": 30.8
}
```

6.5.5 Service cần kết nối

Kết nối với	Mục đích
IoT Ingestion	Lấy dữ liệu cảm biến.
Camera Stream / AI Vision	Lấy dữ liệu phát hiện.
Access Gate	Lấy dữ liệu ra/vào.
Core Business	Lấy dữ liệu cảnh báo/quyết định.

6.5.6 Công cụ gợi ý

- Python + FastAPI.
- Node.js.
- PostgreSQL hoặc TimescaleDB.
- MongoDB.
- Pandas nếu cần xử lý dữ liệu.
- Docker.

6.6 Đề tài 6. Core Business Service

6.6.1 Vai trò

Đây là service trung tâm xử lý luật nghiệp vụ. Service này quyết định khi nào một sự kiện là bình thường, khi nào là bất thường và khi nào cần tạo cảnh báo.

6.6.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận dữ liệu từ IoT, Access Gate và AI Vision.
- Áp dụng rule/policy.
- Tạo alert nếu có bất thường.
- Gửi alert sang Notification.
- Cung cấp dữ liệu cho Analytics.

6.6.3 Rule gợi ý

- Nhiệt độ vượt ngưỡng thì tạo cảnh báo.
- Phát hiện người lạ ở khu vực cấm thì tạo cảnh báo.
- Quẹt thẻ ngoài giờ cho phép thì tạo cảnh báo.
- Một thẻ quẹt quá nhiều lần trong thời gian ngắn thì đánh dấu bất thường.

6.6.4 Đầu ra gợi ý

```
{
  "alert_id": "ALT-001",
  "type": "unauthorized_access",
  "severity": "high",
  "message": "Access attempt outside allowed time"
}
```

6.6.5 Service cần kết nối

Kết nối với	Mục đích
IoT Ingestion	Nhận dữ liệu cảm biến.
Access Gate	Nhận sự kiện ra/vào.
AI Vision	Nhận kết quả phân tích ảnh.
Notification	Gửi cảnh báo.
Analytics	Cung cấp dữ liệu quyết định/cảnh báo.

6.6.6 Công cụ gợi ý

- Node.js + Fastify.

- Python + FastAPI.
- PostgreSQL.
- Rule engine tự viết đơn giản.
- Docker.

Đây là service có nhiều phụ thuộc nhất. Nhóm chọn đề tài này cần chủ động làm việc với các nhóm khác ngay từ Buổi 1.

6.7 Đề tài 7. Notification Service

6.7.1 Vai trò

Service này chịu trách nhiệm gửi thông báo khi hệ thống phát hiện sự kiện bất thường.

6.7.2 Nhiệm vụ chính

Nhóm cần xây dựng một service có khả năng:

- Nhận alert từ Core Business.
- Gửi thông báo qua ít nhất một kênh.
- Ghi log trạng thái gửi.
- Có cơ chế retry hoặc tránh gửi trùng ở mức đơn giản.

6.7.3 Kênh thông báo gợi ý

- Telegram Bot.
- Email.
- Discord webhook.
- Zalo mock.
- SMS mock.
- Console/log nếu không dùng được dịch vụ thật.

6.7.4 Dữ liệu đầu vào gợi ý

```
{
  "alert_id": "ALT-001",
  "severity": "high",
  "message": "Unknown person detected near main gate",
  "target": "security_team"
}
```

6.7.5 Đầu ra mong đợi

```
{
  "sent": true,
  "channel": "telegram",
  "status": "delivered"
}
```

6.7.6 Service cần kết nối

Kết nối với	Mục đích
Core Business	Nhận cảnh báo.
Analytics	Cung cấp log thông báo.
API Gateway nếu có	Nhận request từ hệ thống chung.

6.7.7 Công cụ gợi ý

- Python + FastAPI.
- Node.js.
- Telegram Bot API.
- SMTP Email.
- Webhook.
- Docker.

7 Cấu trúc repository khuyến nghị

Mỗi nhóm cần tổ chức repo rõ ràng để giảng viên và nhóm khác có thể đọc, chạy và kiểm thử.

```
team-service-name/
  README.md
  openapi.yaml
  service_boundary.md
  endpoint_catalog.md
  requirements.txt hoặc package.json
  Dockerfile
  docker-compose.yml          # nếu service có DB/worker/phụ trợ
  .dockerignore
  .env.example
  RUN_LOCAL.md
  src/
    main.py hoặc server.js
    routes/
    services/
    models/
    utils/
  tests/
    postman_collection.json
    environment_local.json
    newman_report.html
```

```
docs/  
  architecture_one_pager.md  
  deployment_diagram.png  
  changelog.md  
  integration_notes.md  
evidence/  
  screenshots/  
  logs/  
  demo_video_link.txt  
  reflection/
```

Một file README.md tối thiểu cần có:

1. Tên service.
2. Vai trò của service trong hệ thống.
3. Thành viên nhóm.
4. Công nghệ sử dụng.
5. Cách chạy local.
6. Cách chạy bằng Docker.
7. Danh sách endpoint chính.
8. Service upstream/downstream.
9. Link Postman Collection hoặc hướng dẫn chạy test.
10. Minh chứng demo.

8 Lộ trình thực hiện theo 6 buổi

8.1 Buổi 1. Chọn đề tài và xác định service boundary

8.1.1 Việc cần làm

- Chia lớp thành 14 nhóm.
- Chọn Product A hoặc Product B.
- Chọn service phụ trách.
- Vẽ sơ đồ tổng quan sản phẩm.
- Xác định service của nhóm mình nhận gì, trả gì, gọi ai, ai gọi mình.

8.1.2 Artefact cần nộp

1. Tên nhóm.
2. Danh sách thành viên.
3. Service đã chọn.
4. Service Boundary Diagram.
5. Bảng “ai gọi ai”.

6. Danh sách endpoint dự kiến.

8.2 Buổi 2. Thiết kế hợp đồng API

8.2.1 Việc cần làm

- Viết `openapi.yaml`.
- Mô tả endpoint, request, response, status code.
- Thống nhất hợp đồng API với nhóm liên quan.
- Chốt phiên bản API đầu tiên.

8.2.2 Artefact cần nộp

1. `openapi.yaml`.
2. Endpoint Catalog.
3. Biên bản chốt API với nhóm liên quan.

8.3 Buổi 3. Kiểm thử tích hợp

8.3.1 Việc cần làm

- Tạo Postman Collection.
- Viết test cho happy path, sai dữ liệu, sai auth và lỗi service.
- Chạy thử với mock server hoặc local server.
- Consumer test API của provider.

8.3.2 Artefact cần nộp

1. Postman Collection.
2. Environment file.
3. Test report.
4. Reliability Checklist.

8.4 Buổi 4. Đóng gói bằng Docker

8.4.1 Việc cần làm

- Viết Dockerfile.
- Tạo `.dockerignore`.
- Tạo `.env.example`.
- Build image.
- Chạy service trong container.

- Chạy lại Postman test trên container.

8.4.2 Artefact cần nộp

1. Dockerfile.
2. .dockerignore.
3. .env.example.
4. RUN_LOCAL.md.
5. Ảnh/log container chạy thành công.

8.5 Buổi 5. Kết nối nhiều service

8.5.1 Việc cần làm

- Viết `docker-compose.yml`.
- Cấu hình network giữa các service.
- Kết nối thử với ít nhất một nhóm khác.
- Chạy một luồng end-to-end ngắn.

8.5.2 Artefact cần nộp

1. `docker-compose.yml`.
2. Deployment Diagram.
3. Minh chứng kết nối với nhóm khác.
4. Log hoặc screenshot luồng chạy thành công.

8.6 Buổi 6. Demo Day – Plug-a-thon

8.6.1 Việc cần làm

- Demo service của nhóm.
- Kết nối với service khác trong cùng Product.
- Trả lời phản biện kiến trúc.
- Chứng minh service chạy được từ repo sạch.
- Nộp Demo Pack cuối cùng.

8.6.2 Artefact cần nộp

1. Demo Pack hoàn chỉnh.
2. Video demo.
3. Slide kiến trúc.

9 Gợi ý phân vai trong nhóm

Mỗi nhóm 3–4 sinh viên nên chia vai rõ ràng.

Vai trò	Nhiệm vụ
Service Lead	Nắm yêu cầu, điều phối nhóm, làm việc với nhóm khác.
API/Contract Owner	Viết <code>openapi.yaml</code> , quản lý endpoint và schema.
Backend Developer	Code service chính.
Test & DevOps Owner	Postman, Dockerfile, Compose, log và minh chứng.

Nếu nhóm có 3 sinh viên thì một bạn có thể kiêm hai vai. Tuy nhiên, nhóm vẫn phải chứng minh trong reflection cá nhân rằng từng thành viên có đóng góp thật.

10 Luật làm việc giữa các nhóm

10.1 Luật 1. Không làm service cô lập

Mỗi nhóm phải xác định ít nhất:

- Một service mình gọi đến, hoặc
- Một service sẽ gọi mình, hoặc
- Một nguồn dữ liệu/event mình cung cấp cho hệ thống.

10.2 Luật 2. Không đổi API âm thầm

Nếu đổi API mà không báo nhóm liên quan, lỗi phát sinh trong tích hợp sẽ tính vào nhóm đổi API.

Khi cần đổi API, nhóm phải:

1. Thông báo cho nhóm liên quan.
2. Cập nhật `openapi.yaml`.
3. Ghi rõ thay đổi trong `changelog.md`.
4. Được nhóm consumer đồng ý.

10.3 Luật 3. Không hard-code endpoint của nhóm khác

Các URL, token, host và port phải để trong file `.env` hoặc `.env.example`.

Ví dụ:

```
CORE_SERVICE_URL=http://core-business:8000
AI_VISION_URL=http://ai-vision:8000
NOTIFICATION_URL=http://notification:8000
```

10.4 Luật 4. Repo phải chạy lại được

Một repo tốt cần có hướng dẫn kiểu:

```
docker compose up --build
```

hoặc:

```
docker build -t team-service .
docker run --env-file .env.example -p 8000:8000 team-service
```

Nếu giảng viên hoặc nhóm khác clone repo về mà không biết chạy như thế nào, nhóm bị trừ điểm phần công nghệ nền tảng.

10.5 Luật 5. Front-end không phải trọng tâm

Học phần này không chấm nặng giao diện. Nhóm có thể làm UI, nhưng không được bỏ qua các phần chính:

- API.
- OpenAPI.
- Postman test.
- Docker.
- Kết nối service.
- Minh chứng vận hành.

11 Demo Pack cuối kỳ

Mỗi nhóm nộp một **Demo Pack** gồm các minh chứng sau:

Thành phần	Bắt buộc
README mô tả service	Có
Service Boundary Diagram	Có
openapi.yaml	Có
Endpoint Catalog	Có
Postman Collection	Có
Test report Newman hoặc ảnh minh chứng test	Có
Source code	Có
Dockerfile	Có
.env.example	Có
docker-compose.yml nếu service có DB/worker/phụ trợ	Có
Log hoặc ảnh service chạy thành công	Có
Minh chứng kết nối với service khác	Có
Slide kiến trúc	Có
Video demo ngắn 2–3 phút	Có

12 Tiêu chí đánh giá

Tiêu chí	Trọng số	Mô tả
----------	----------	-------

Phân tích service boundary	10%	Xác định đúng vai trò, input, output, upstream và downstream.
Hợp đồng API	20%	openapi .yaml rõ ràng, đúng logic, có schema, có ví dụ.
Kiểm thử tích hợp	15%	Có Postman Collection, test case đủ loại, có report.
Công nghệ nền tảng	20%	Dockerfile, Compose, env, healthcheck, chạy được từ repo sạch.
Kết nối liên service	20%	Bắt tay được với service khác, có minh chứng end-to-end.
Trình bày và bảo vệ	15%	Giải thích được quyết định kiến trúc, trả lời phản biện.

Điểm nhấn quan trọng:

Nhóm không được điểm cao chỉ vì code nhiều. Nhóm được điểm cao khi service có thiết kế rõ, hợp đồng tốt, test được, đóng gói được và kết nối được.

13 Rubric chi tiết

Tiêu chí	Mức điểm	Minh chứng	Gợi ý đánh giá
Service Boundary	Tốt	Sơ đồ rõ, xác định đủ input/output/upstream/downstream/data owner.	Nhóm hiểu đúng vai trò service trong hệ thống.
Service Boundary	Đạt	Có sơ đồ nhưng còn thiếu một số quan hệ.	Nhóm cần bổ sung quan hệ với service khác.
Service Boundary	Chưa đạt	Sơ đồ mơ hồ, service bị cô lập.	Không chứng minh được vai trò trong sản phẩm.
OpenAPI Contract	Tốt	Có schema, status code, example, error model, auth.	Consumer có thể dùng spec để gọi thử.
OpenAPI Contract	Đạt	Có endpoint chính nhưng thiếu example hoặc error model.	Vẫn dùng được nhưng chưa chuyên nghiệp.
OpenAPI Contract	Chưa đạt	Không có openapi .yaml hoặc spec sai nhiều.	Không đủ điều kiện tích hợp.
Testing	Tốt	Có happy path, negative, auth, boundary, report.	Test thể hiện được độ tin cậy.
Testing	Đạt	Có test cơ bản nhưng chưa đủ phủ lỗi.	Cần bổ sung các case sai dữ liệu/lỗi upstream.
Testing	Chưa đạt	Không có test hoặc chỉ test thủ công bằng trình duyệt.	Không đủ minh chứng kiểm thử.
Docker/Compose	Tốt	Chạy được từ repo sạch, có env, healthcheck, compose.	Đúng tinh thần công nghệ nền tảng.
Docker/Compose	Đạt	Có Dockerfile nhưng hướng dẫn còn thiếu.	Cần bổ sung RUN_LOCAL.md.
Docker/Compose	Chưa đạt	Không chạy được bằng Docker.	Không đạt yêu cầu nền tảng.
Integration	Tốt	Kết nối được với ít nhất một service khác và có bằng chứng.	Có log, screenshot hoặc video end-to-end.

Integration	Đạt	Có mô phỏng kết nối nhưng chưa ổn định.	Cần giải thích rõ điểm chưa hoàn thành.
Integration	Chưa đạt	Service chạy đơn lẻ, không kết nối được.	Không đạt mục tiêu bài tập lớn.

14 Kết luận

Bài tập lớn này không yêu cầu sinh viên phải làm một hệ thống hoàn hảo như doanh nghiệp, nhưng yêu cầu sinh viên học đúng tư duy của hệ thống hiện đại:

- Một service phải có ranh giới rõ.
- Một API phải có hợp đồng.
- Một hệ thống phải test được.
- Một ứng dụng phải đóng gói được.
- Một sản phẩm phải kết nối được nhiều phần độc lập.

Đến cuối học phần, mỗi nhóm không chỉ có một bài nộp, mà có một service có thể đưa vào GitHub portfolio để chứng minh rằng mình đã từng làm việc với tư duy microservice, API contract, Docker và tích hợp hệ thống.